

FINANCIAL RESEARCH AI AGENT DEVELOPMENT

Kollipati Lakshmi Sravanthi
P Sai Lekhya
Kalisetty Naga Sathya
Hemashree

*Indian Stock Analysis
Assistant*

1

Financial API integration strategies and challenges

2

Technical architecture decisions for handling market data

3

Risk assessment and compliance considerations

4

Individual team member contributions and learning outcomes

5

Future improvement ideas for enhanced financial features

TABLE OF CONTENT

FINANCIAL API INTEGRATION STRATEGIES AND CHALLENGES

APIs Used in Our Project

Yahoo Finance (yfinance) — Real-time & historical stock prices
(NSE/BSE), fundamentals, market cap, P/E, EPS, etc.

NewsAPI — Latest news articles per company for sentiment analysis

Google Gemini (via LangChain) — AI-powered financial Q&A assistant



Integration Strategies

Caching with `@st.cache_data`

Stock data (TTL: 5 min) and sparkline batches (TTL: 15 min) are cached to reduce redundant API calls and avoid rate limits.

Fallback mechanisms

If `ticker.history()` returns empty, we fall back to `yf.download()` with adjusted parameters — ensures data still loads even if one method fails.

Symbol validation before fetching

`validate_stock_symbol()` checks if a ticker is valid before making expensive calls, and suggests `.NS/.BO` suffixes if missing.

Batch requests for efficiency

Watchlist sparklines are fetched in a single batch call for multiple symbols instead of one-by-one — reduces API load significantly.

Graceful degradation for missing data

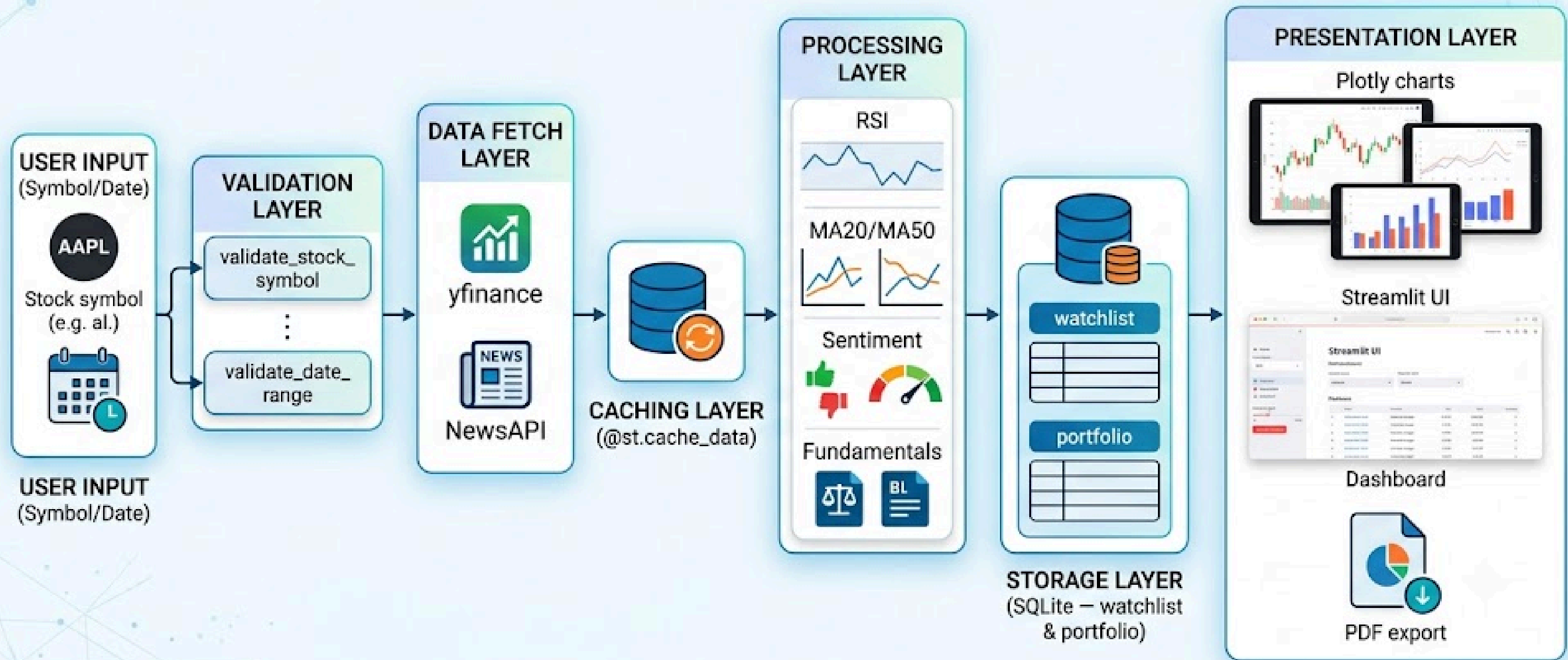
Fundamentals use a helper `fmt()` function that returns "N/A" instead of crashing when yfinance returns `None` for Indian stocks (common issue).

Environment-based API key management

NewsAPI and Gemini keys are loaded via `.env` and `os.getenv()`, never hardcoded — keeps credentials secure and configurable per deployment.

Challenge	Solution
yfinance returns empty data for some Indian stocks	Added fallback to yf.download() + clear error messages
Rate limiting on repeated requests	Implemented caching (ttl=300/ttl=900)
NewsAPI free tier limits (100 req/day)	Cached results for 10 minutes (ttl=600)
Inconsistent data during market closed hours (intraday)	Built market-hours checker; show contextual error banners
Unicode/currency symbols (₹) breaking PDF generation	Custom clean_rupee() function sanitizes text before PDF export
Gemini API timeouts	Added timeout=30 and max_retries=1 to prevent UI freeze

STOCK ANALYSIS APPLICATION ARCHITECTURE: USER TO PRESENTATION FLOW



Modular separation of concerns

Three distinct modules: **app.py** (UI/routing), **week56_features.py** (data + DB logic), **pdf_generator.py** (export logic) — keeps code maintainable and testable.

SQLite for persistence

Chosen over in-memory storage so watchlist and portfolio survive app restarts. Two tables: watchlist and portfolio, each with `init_db()` functions for safe schema creation.

Session state for UI continuity

st.session_state holds current symbol, period, fetched history, and theme — avoids redundant API calls on every widget interaction/rerun (a core Streamlit challenge).

Computed indicators on top of raw data

Technical indicators (RSI, MA20, MA50) are calculated client-side using pandas (`add_technical_indicators()`) rather than relying on a paid indicators API — keeps cost at zero.

Theme-aware visualization layer

Plotly chart functions accept a `theme_mode` parameter, dynamically adjusting colors/templates — separates data logic from presentation logic.

Market-hours-aware data fetching

get_market_status() runs independently of stock data fetching, using `pytz` for IST conversion — informs UI banners and explains data gaps without blocking the main data flow.

Layered caching strategy

get_stock_data() → 5-min TTL (price changes frequently)
get_sparkline_data_batch() → 15-min TTL (less critical, used for trends)
get_news_sentiment() → 10-min TTL (news doesn't update every second)
Different TTLs based on how time-sensitive each data type is.

Defensive error handling at every layer

Every API call (yfinance, NewsAPI, Gemini) is wrapped in `try/except` with fallback values ("N/A", empty lists, None) — ensures one failed API never crashes the whole dashboard.

RISK ASSESSMENT & COMPLIANCE CONSIDERATIONS

1. "Not Investment Advice" disclaimers everywhere
 - Footer of every PDF report: "For educational use only. Not investment advice."
 - AI Chatbot system prompt explicitly instructs Gemini: "Always add a disclaimer that this is not investment advice."
 - Landing page footer: "Educational Platform • For research purposes only."
2. No execution capability
3. The app is read-only/analytical — it never places trades, connects to brokerage accounts, or handles real money. This avoids SEBI broker-dealer regulations entirely.
4. Encouraging professional consultation
5. PDF summary explicitly states: "Please consult a SEBI registered financial adviser before making any investment decisions."

FINANCIAL RISK INDICATORS WE SURFACE (TO THE USER)

Metric	What It Tells the User
RSI (14-day)	Overbought/Oversold — momentum risk
Beta	Volatility relative to the market
Debt/Equity Ratio	Leverage/financial risk
Annualized Volatility (Sector Comparison)	Price fluctuation risk
Max Drawdown (Sector Comparison)	Worst-case historical loss from peak
News Sentiment Score	Qualitative market mood risk

These give users a risk-informed view, not a buy/sell signal.

Data Handling & Security Considerations

API keys never hardcoded — stored in .env, loaded via `os.getenv()`, excluded from GitHub via .gitignore.

No personal financial data collected — portfolio data is stored locally in SQLite, not transmitted to third parties beyond price lookups.

Graceful failure over silent errors — if data can't be validated, the user sees a clear warning rather than potentially misleading "N/A" treated as zero.

Input validation prevents malformed data (negative quantities/prices, invalid date ranges) from corrupting calculations or the database.

Known Limitations We Acknowledge

Data is sourced from free-tier APIs (yfinance, NewsAPI) — not real-time exchange-grade data, may have delays.

Sentiment analysis uses TextBlob (lexicon-based), which is simpler than transformer-based models — may misclassify nuanced financial language.

The app does not account for taxation, brokerage fees, or slippage in portfolio return calculations.

INDIVIDUAL TEAM MEMBER CONTRIBUTIONS AND LEARNING OUTCOMES

Member	Contribution	Key Learning
P Sai Lekhya	API integration (Weeks 1-4) — stock data, news sentiment, RSI/MA indicators, caching	Handling unreliable financial APIs & rate limits
Kalisetty Naga Sathya	Domain features (Weeks 5-6) — watchlist, fundamentals, sector comparison, portfolio DB + deployment	SQLite design & Streamlit Cloud deployment
Kollipati Lakshmi Sravanthu	UI/UX design + README & architecture docs	Building polished UI with custom CSS; technical writing
Hemashree	PDF report module + compliance text review	PDF generation (FPDF) & fintech compliance language

FUTURE IMPROVEMENTS

- Real-time WebSocket price updates instead of polling
- More technical indicators (MACD, Bollinger Bands)
- Multi-user authentication for personalized portfolios
- Integration with broker APIs (Zerodha Kite) for live trading data
- Push notifications for price alerts



**THANK
YOU**